

MLOps Midterm Revision – Complete Summary

1 What is MLOps & Why Do We Need It?

The Problem

In traditional ML, models often work perfectly in Jupyter notebooks but fail when deployed in the real world.

Why? Because:

- The **data pipeline** in production may differ from the one used for training.
- The **model isn't versioned or monitored**.
- Manual steps make it impossible to reproduce results.

The Solution → MLOps

MLOps = Machine Learning + DevOps

It applies software engineering principles (automation, CI/CD, testing, monitoring) to machine learning systems.

DevOps philosophy:

"If it's not automated, it's broken."

MLOps adopts this: every step (from data to deployment) should be automated, reproducible, and monitored.

MLOps Lifecycle

1. Data Management:

- Collect, clean, and preprocess data.
- Version control datasets using tools like DVC.

2. Experiment Tracking:

- Record which data, model, and parameters were used.
- Tools: MLflow, Weights & Biases.

3. Model Registry:

- Keep track of model versions (v1, v2, etc.).
- Helps rollback to older versions if new one fails.

4. Deployment:

- Convert the model into a service (often using Docker + FastAPI).
- Deploy on cloud (AWS, Azure, GCP).

5. Monitoring:

- Watch metrics like accuracy, latency, data drift.
- Retrain automatically if performance drops.

2 Virtual Environments (venv & Conda)

The Problem: Dependency Hell

Every project has its own library versions. Installing everything globally causes version conflicts.

Virtualenv

- Built into Python.
- Creates isolated folders that contain their own `python` and libraries.

Commands:

```
python -m venv myenv
source myenv/bin/activate # (Mac/Linux)
myenv\Scripts\activate   # (Windows)
```

deactivate

Conda

- Part of Anaconda distribution.
- Handles both Python and non-Python packages (like CUDA).
Commands:

```
conda create --name mlops python=3.8
conda activate mlops
conda install numpy pandas
conda env export > environment.yml
conda env create -f environment.yml
conda deactivate
```

Why Conda is useful: It ensures reproducibility across OS and teams.

3 Virtual Machines (VMs) vs Containers

Virtual Machines

- Virtualize **hardware** using a **hypervisor**.
- Each VM has its own OS, kernel, and resources.
- Examples: VMware, VirtualBox.
Pros: Full isolation, secure, stable.
Cons: Heavy, slow startup, duplicates resources.

Containers

- Virtualize **operating system**, not hardware.
- Share the host's kernel.
- Example: Docker containers.
Pros: Lightweight, fast startup (seconds), portable, ideal for microservices.

Cons: Less isolation than VMs (share kernel).

Key difference:

- VM = full computer inside your computer.
- Container = lightweight isolated process sharing host kernel.

Docker – The Core of MLOps Deployment

Docker = tool that packages code + dependencies + environment into **containers**.
Think of it as a “**portable box**” that runs identically on any system.

Basic Docker Commands

Action	Command	Explanation
Build image	<code>docker build -t myapp:v1 .</code>	Builds image from Dockerfile
Run container	<code>docker run -it myapp:v1</code>	Runs a container
List containers	<code>docker ps -a</code>	Shows all containers
Stop container	<code>docker stop <id></code>	Stops running container
Remove container	<code>docker rm <id></code>	Deletes container
Push to DockerHub	<code>docker push username/myapp:v1</code>	Upload image to registry

Dockerfile Basics

Example:

```
FROM centos:8
RUN dnf install -y python38
ENTRYPOINT ["/bin/bash"]
```

Explanation:

- **FROM** = base image (defines OS layer)
- **RUN** = commands executed during build (creates a layer)
- **ENTRYPOINT** = default command when container starts

Layer concept: Each instruction forms a new layer → cached for faster builds.

Docker Terms

- **Image:** Blueprint for containers (like a recipe).
 - **Container:** Running instance of an image (like a cake baked from recipe).
 - **Registry:** Remote store for images (e.g., Docker Hub).
 - **Docker Daemon:** Background service managing containers.
-

5 Kubernetes (Container Orchestration)

When you have **many containers**, you need a system to manage them → **Kubernetes (K8s)**.

Purpose

- Automates **deployment, scaling, load balancing**, and **health checks** for containers.

Scaling

- **Vertical scaling:** Increase CPU/RAM of a node.
- **Horizontal scaling:** Add more pods/nodes (preferred for cloud apps).

Kubernetes Components

- **Pod:** Smallest deployable unit (one or more containers).
 - **Node:** A physical/virtual machine running pods.
 - **Cluster:** Group of nodes managed by the K8s master.
 - **Controller:** Maintains desired state (e.g., keeps 3 replicas running).
-

6 Feature Stores

The Problem: Train-Serve Skew

Model performs well in training but poorly in production because data pipelines differ.
For example:

- Training calculates “average purchase per month” differently than the live app.

Solution: Feature Store

A **Feature Store** is a centralized registry for **consistent, reusable ML features**.

Functions:

1. Store curated, validated features.
2. Serve features consistently in both **training** and **inference**.
3. Detect **data drift** over time.
4. Allow sharing of features across teams.

Examples: Uber’s Michelangelo, Tecton, Vertex AI Feature Store, SageMaker Feature Store.

7 AutoML (Automated Machine Learning)

AutoML automates all model development steps:

- **Data preprocessing**
- **Feature engineering**
- **Model selection**
- **Hyperparameter tuning**

Benefits:

- Saves time for baseline models.
- Useful for non-experts to get decent results.
- Allows fair comparisons before custom tuning.

Tools:

- **Open Source:** Ludwig (by Uber), FLAML (by Microsoft)
- **Cloud:** SageMaker Autopilot (AWS), Vertex AI AutoML (GCP), Azure AutoML.

AutoML doesn't remove human role — it accelerates prototyping.

8 Cloud ML Platforms (MLOps on Cloud)



AWS

- **Compute:** EC2 (VMs), ECS (Docker), EKS (Kubernetes), Fargate (serverless).
- **Storage:** S3, EFS, Lake Formation.
- **Pipeline:** Glue, Kinesis, Data Pipeline.
- **Feature Store:** SageMaker Feature Store.

- **Model Deployment:** SageMaker Endpoints, Bedrock for LLMs.
 - **Monitoring:** CloudWatch, Model Monitor.
 - **Security:** IAM, KMS.
-

Azure

- **Compute:** Azure ML Compute Cluster, AKS (Kubernetes), Container Apps.
 - **Storage:** Blob Storage, Data Lake Gen2.
 - **Feature Store:** Azure ML Feature Store (preview).
 - **Deployment:** Azure ML Endpoints, Azure OpenAI.
 - **Monitoring:** Data Drift Monitor, Application Insights.
 - **Security:** RBAC, Key Vault.
-

GCP

- **Compute:** Vertex AI Training, Cloud Run, GKE Autopilot.
 - **Storage:** Cloud Storage, BigLake.
 - **Feature Store:** Vertex AI Feature Store.
 - **Deployment:** Vertex AI Endpoints.
 - **Monitoring:** Vertex Model Monitoring, Cloud Logging.
 - **Security:** IAM, VPC Service Controls.
-

MLflow – Experiment Tracking & Model Registry

MLflow helps track, reproduce, and compare experiments.
It has 4 main components:

1. **Tracking:** Logs parameters, metrics, and artifacts.
2. **Projects:** Defines reproducible code environments.
3. **Models:** Standardizes model packaging (e.g., scikit-learn, PyTorch).
4. **Registry:** Central place for approved model versions.

Example Code:

```
import mlflow

mlflow.set_tracking_uri("http://localhost:5000")
with mlflow.start_run():
    mlflow.log_param("alpha", 0.5)
    mlflow.log_metric("rmse", 0.23)
```

10 Drift Detection

Types

1. **Data Drift:** Input data distribution changes (e.g., age range shifts).
2. **Concept Drift:** Relationship between features and target changes.

Example:

A model predicting loan approval fails when economy shifts → target logic changes.

Tools:

Evidently AI, AWS Model Monitor, Azure Data Drift Monitor.

Solution:

Continuous monitoring → trigger retraining when drift exceeds threshold.

11 CI/CD in MLOps

Continuous Integration (CI)

- Automatically build and test every code change.
- Tools: GitHub Actions, Jenkins, GitLab CI.
Example: test model accuracy every time code is pushed.

Continuous Delivery/Deployment (CD)

- Automatically deploy code/model to production after CI passes.
- Uses Infrastructure as Code (IaC) for consistency.

Benefits:

- Faster updates, fewer manual errors, reproducible builds.
-

Infrastructure as Code (IaC)

Everything about your infrastructure is defined in code (Terraform, CloudFormation).
This makes environments consistent and reproducible.

Microservices & FastAPI

A **microservice** = a small, independent service doing one task (e.g., model prediction).

Why microservices?

- Easier to scale, test, and deploy individually.
- Common in MLOps to separate “model service” from “data ingestion service.”

Example with FastAPI:

```
from fastapi import FastAPI
import joblib
```

```
app = FastAPI()
model = joblib.load("model.pkl")
```

```
@app.post("/predict")
def predict(data: dict):
    return {"prediction": model.predict([data["values"]]).tolist()}
```

Dockerize this FastAPI app → push to registry → deploy on cloud.

13 Git & Version Control Basics

Git keeps track of **code versions**, branches, and collaboration history.

Core Commands

```
git init
git add .
git commit -m "message"
git branch feature1
git checkout -b feature1
git push origin main
git pull origin main
git merge feature1
```

Why it matters:

MLOps uses Git for both **code** and **infrastructure (IaC)**, ensuring full reproducibility.

14 Responsible AI, Security, and Observability (LLMOps intro)

- **Observability:** Track cost, latency, and token usage (especially for LLMs).
- **Guardrails:** Filter toxicity, bias, and prompt injection.
- **Tools:** LangSmith, Arize Phoenix, Bedrock Guardrails.

- **LLM CI/CD:** Quality gates prevent bad prompts or responses from going live.



Final Revision Summary Table

Topic	Tool / Command	Key Idea
Environment Mgmt	<code>conda create, env export</code>	Reproducibility
Containers	<code>docker build, FROM, ENTRYPOINT</code>	Portability
Orchestration	Kubernetes	Scale & manage containers
Feature Store	Vertex/SageMaker	Avoid train-serve skew
AutoML	FLAML, Ludwig	Automate ML pipeline
Drift Detection	Evidently, Model Monitor	Maintain performance
Experiment Tracking	MLflow	Reproduce & compare runs
CI/CD	GitHub Actions	Automate testing + deployment
Cloud MLOps	AWS, Azure, GCP	Managed services for ML
FastAPI	<code>/predict</code> endpoint	Serve models via REST